

IFT605/IFT713 — Systèmes Répartis et Multi-Agents
Université de Sherbrooke | Automne 2026 | Enseignante : Amal Ben Abdellah

Projet — CryptoLedger

Implémentation d'une Mini-Chaîne de Blocs

Fiche du projet

Durée estimée 3 semaines (option : 4 semaines)

Environnement Java 21 (Eclipse Temurin LTS) • Maven 3.9.x

Type Projet d'équipe — 2 ou 3 étudiants

Technologies Java 21 • Maven 3.9.x • JUnit 5

Chapitres Chap. 1 → 5 (Cryptographie, Décentralisation, Mécanismes, Stockage)

Table des matières

Mise en contexte	3
Objectifs pédagogiques	3
Description du système à implémenter	4
Module 1 — Fondations cryptographiques	5
Module 2 — Structure de la blockchain	6
Module 3 — Modèle de transactions (UTXO)	7
Module 4 — Consensus et minage (PoW)	8
Module 5 — Réseau P2P et portefeuille	9
Module 6 — Fonctionnalités avancées (4e semaine optionnelle)	10
Livrables attendus	11
Calendrier recommandé	12
Critères d'évaluation	13
Ressources et références	14

1 Mise en contexte

Bitcoin est bien plus qu'une cryptomonnaie : c'est une démonstration concrète des principes fondamentaux des systèmes répartis — consensus distribué, tolérance aux fautes byzantines, communication P2P, et sécurité cryptographique sans autorité centrale.

Dans ce projet, vous allez implémenter CryptoLedger, un système de paiement décentralisé simplifié inspiré de Bitcoin. Chaque module du projet correspond directement à un chapitre du cours :

Module	Chapitre de cours	Thème
1 — Cryptographie	Chap. 2 — Bitcoin et Cryptographie	SHA-256, ECDSA, engagements
2 — Blockchain	Chap. 2 — Bitcoin et Cryptographie	Blocs, arbres de Merkle, immuabilité
3 — Transactions	Chap. 4 — Bitcoin Mécanismes	UTXO, scripts P2PKH
4 — Consensus	Chap. 3 — Bitcoin et la Décentralisation	Preuve de travail, double dépense
5 — P2P + Portefeuille	Chap. 1, 4, 5	Gossip, wallet, clés HD
6 — Avancé (optionnel)	Chap. 5 — Stocker et Utiliser les Bitcoins	Multisig, partage de secret

2 Objectifs pédagogiques

À la remise du projet, chaque équipe sera capable de :

- Implémenter et valider les six propriétés cryptographiques de SHA-256 (résistance aux collisions, dissimulation, puzzle-friendliness...).
- Construire une chaîne de blocs tamper-evident avec des pointeurs de hachage et un arbre de Merkle fonctionnel.
- Modéliser le cycle de vie complet d'une transaction Bitcoin (UTXO, signatures, vérification des entrées/sorties).
- Implémenter un mécanisme de Preuve de Travail (PoW) avec ajustement de la difficulté et simulation d'une attaque de double dépense.
- Faire communiquer plusieurs nœuds Java via un protocole gossip P2P (sockets TCP).
- Relier tous les mécanismes dans un système intégré démontrable en classe.
- (Optionnel) Implémenter les multisignatures M-of-N et le partage de secret de Shamir.

3 Description du système à implémenter

CryptoLedger est un réseau local de nœuds Java simulant les fonctionnalités essentielles de Bitcoin.

L'architecture cible est la suivante :

```
cryptoledger-parent/    ← POM parent Maven
├─ crypto/              ← Module 1 : primitives cryptographiques
└─ blockchain/          ← Module 2 : blocs + Merkle tree
```

└─ transactions/	← Module 3 : UTXO + scripts
└─ consensus/	← Module 4 : PoW + minage
└─ network/	← Module 5 : nœuds P2P + portefeuille
└─ demo/	← Scénarios de démonstration intégrés

i Note : Langage imposé : Java 21.

Build : Maven 3.9.x (projet multi-modules).

Tests : JUnit 5 — au moins 3 tests unitaires par module.

Pas de bibliothèque Bitcoin externe autorisée (bouncy castle OK pour ECDSA).

Module 1 — Fondations cryptographiques

Référence cours : Chap. 2 — Bitcoin et Cryptographie (diapositives 3–22).

1.1 Fonctions de hachage

Implémentez la classe `CryptoUtils` offrant :

1. `sha256(byte[] data) → byte[]` — wrapper sur `java.security.MessageDigest`.
2. `sha256Hex(String input) → String` — version hexadécimale lisible.
3. Démontrez les 6 propriétés du cours : entrée variable, taille fixe, déterminisme, résistance aux collisions, résistance à la préimage, effet avalanche.

1.2 Engagement cryptographique

Implémentez `commit(String msg, String nonce) = SHA-256(nonce || msg)` et `verify(String commit, String msg, String nonce) → boolean`.

Scénario de test : simulez le jeu de pile ou face (Diapo 7, Chap. 2).

```
// Alice s'engage
String nonce = CryptoUtils.randomNonce();
String commit = CryptoUtils.commit("PILE", nonce);
// Bob devine, Alice révèle
boolean ok = CryptoUtils.verify(commit, "PILE", nonce); // true
boolean ko = CryptoUtils.verify(commit, "FACE", nonce); // false
```

1.3 Paires de clés et signatures numériques (ECDSA)

Utilisez l'algorithme EC avec la courbe P-256 (ou secp256k1 via Bouncy Castle) :

1. `KeyPair generateKeyPair()` — génère (sk, pk).
2. `byte[] sign(byte[] data, PrivateKey sk)` — signature ECDSA.
3. `boolean verify(byte[] data, byte[] sig, PublicKey pk)` — vérification.
4. `String publicKeyToAddress(PublicKey pk)` — adresse = Base58(SHA-256(pk)).

✓ **Résultat attendu** : Test attendu :

```
KeyPair kp = CryptoUtils.generateKeyPair();
byte[] sig = CryptoUtils.sign("hello".getBytes(), kp.getPrivate());
assert CryptoUtils.verify("hello".getBytes(), sig, kp.getPublic()); // OK
assert !CryptoUtils.verify("world".getBytes(), sig, kp.getPublic()); // KO
```

Module 2 — Structure de la blockchain

Référence cours : Chap. 2 — Bitcoin et Cryptographie (diapositives 10–15).

2.1 Arbre de Merkle

Implémentez `MerkleTree<T>` permettant de :

1. Construire l'arbre depuis une liste de transactions (feuilles = $\text{SHA-256}(\text{tx})$).
2. Retourner la racine de Merkle (merkleRoot).
3. Générer une preuve d'appartenance : `List<byte[]> getMerkleProof(int txIndex)`.
4. Vérifier une preuve : `boolean verifyProof(byte[] txHash, List<byte[]> proof, byte[] root)`.

i Note : Complexité cible : $O(\log n)$ pour la preuve et la vérification.
Cas particulier : si le nombre de feuilles est impair, dupliquer la dernière.

2.2 Bloc et chaîne de blocs

Implémentez les classes :

1. `BlockHeader` : version, previousHash, merkleRoot, timestamp, nonce.
2. `Block` : header + liste de transactions + méthode `computeHash()`.
3. `Blockchain` : `addBlock(Block b)`, `isValid()`, `getBlock(int height)`, `getLatestBlock()`.

Propriété tamper-evident : si on modifie une transaction dans un bloc passé, la chaîne entière à partir de ce bloc doit devenir invalide.

✓ **Résultat attendu** : Test attendu (modification détectée) :

```
Blockchain chain = new Blockchain();
chain.addBlock(b1); chain.addBlock(b2);
assert chain.isValid(); // OK
b1.getTransactions().get(0).tamper(); // modification frauduleuse
assert !chain.isValid(); // chaîne corrompue détectée
```

Module 3 — Modèle de transactions (UTXO)

Référence cours : Chap. 4 — Bitcoin Mécanismes (diapositives 3–13).

3.1 Structure d'une transaction

Implémentez :

1. TransactionInput : txId de référence, index de la sortie (UTXO), scriptSig (signature + pubKey).
2. TransactionOutput : montant (en satoshis), scriptPubKey (adresse destinataire).
3. Transaction : id = SHA-256(inputs + outputs), liste d'inputs, liste d'outputs, frais de transaction.

3.2 UTXO Set et vérification

1. UTXOSet : structure qui maintient toutes les sorties non dépensées.
2. addTransaction(Transaction tx) — ajoute les nouvelles sorties, retire les dépensées.
3. boolean isValid(Transaction tx) — vérifie :
 - a) toutes les entrées référencent des UTXOs existants
 - b) somme(inputs) $\geq$ somme(outputs)
 - c) chaque signature est valide par rapport à la clé publique de l'UTXO

3.3 Script P2PKH simplifié

Implémentez l'exécution d'un script Pay-to-PubKeyHash en deux phases (Diapo 11, Chap. 4) :

```
// Phase 1 — scriptSig (fourni par le dépenseur)
// <signature> <pubKey>
// Phase 2 — scriptPubKey (fourni par le créateur de l'UTXO)
// OP_DUP OP_HASH160 <adresse> OP_EQUALVERIFY OP_CHECKSIG

// Opérations à implémenter sur une pile (Stack<byte[]>) :
// OP_DUP, OP_HASH160, OP_EQUALVERIFY, OP_CHECKSIG
```

Module 4 — Consensus et minage (PoW)

Référence cours : Chap. 3 — Bitcoin et la Décentralisation (diapositives 13–15) et Chap. 4 — Bitcoin Mécanismes.

4.1 Preuve de Travail

1. Implémenter Miner.mine(Block b, int difficulty) → Block miné.
2. La condition PoW : SHA-256(SHA-256(header)) doit commencer par difficulty zéros hexadécimaux.
3. Itérer sur le nonce ($0 \rightarrow 2^{32}$) jusqu'à trouver un hash valide.
4. Mesurer et afficher le temps de minage et le nombre d'itérations.

4.2 Ajustement de la difficulté

Tous les 10 blocs, ajustez la difficulté pour maintenir un temps de minage cible de 10 secondes (simulé) :

```
int newDifficulty = currentDifficulty;
if (actualTime < targetTime / 2) newDifficulty++;
if (actualTime > targetTime * 2) newDifficulty = Math.max(1, newDifficulty - 1);
```

4.3 Simulation de double dépense

Simulez une tentative d'attaque de double dépense (Diapo 10, Chap. 3) :

1. Alice envoie 10 BTC à Bob ET tente simultanément d'envoyer les mêmes 10 BTC à Charlie (double dépense).
2. Simulez deux mineurs compétitifs : l'un mine la transaction vers Bob, l'autre la transaction vers Charlie.
3. Démontrez que la chaîne la plus longue l'emporte et que l'attaque échoue si l'attaquant contrôle moins de 50% de la puissance de calcul.

✓ **Résultat attendu** : Résultat attendu :

Mineur honnête (70%) : mine bloc #5 avec tx BTC→Bob

Attaquant (30%) : tente de miner bloc alternatif avec tx BTC→Charlie

→ Après 6 confirmations, la chaîne honnête est acceptée

→ L'attaquant a dépensé de l'énergie de calcul sans récompense

Module 5 — Réseau P2P et portefeuille

Référence cours : Chap. 1 (intro P2P), Chap. 4 — Mécanismes (diapositives 25–34), Chap. 5 — Stocker et Utiliser les Bitcoins.

5.1 Nœuds P2P (protocole Gossip)

1. Implémenter Node : serveur TCP qui écoute sur un port configurable.
2. Chaque nœud maintient une liste de pairs connus (max 8 connexions actives).
3. Implémenter le protocole de diffusion Gossip/Flooding (Diapo 28, Chap. 4) :
 - Quand une nouvelle transaction ou un nouveau bloc arrive, le diffuser à tous les pairs sauf à celui qui l'a envoyé.
 - Mémoriser les messages déjà traités (par hash) pour éviter les boucles.
4. Implémenter la procédure de connexion (Diapo 26, Chap. 4) :
 - Contacter le nœud d'amorçage (seed node), obtenir la liste des pairs.

5.2 Portefeuille simple

1. Wallet : génère et stocke des paires de clés ECDSA.
2. `getBalance()` → long — calcule le solde en interrogeant l'UTXO set local.
3. `createTransaction(String toAddress, long amount)` → Transaction — construit et signe une transaction depuis les UTXOs disponibles du portefeuille.
4. `broadcast(Transaction tx)` — diffuse la transaction au réseau P2P.

5.3 Scénario de démonstration intégré

Le module `demo/` doit lancer 3 nœuds locaux (ports 6001, 6002, 6003) et démontrer le scénario complet :

- Alice (nœud 6001) mine le bloc genesis et reçoit la récompense de 50 BTC.
- Alice envoie 10 BTC à Bob (nœud 6002) — transaction diffusée par gossip.
- Bob envoie 5 BTC à Charlie (nœud 6003) — enchaînement de transactions.
- Un 4e nœud se connecte après coup et synchronise la chaîne depuis ses pairs.
- Tentative de double dépense détectée et rejetée.

✓ **Résultat attendu** : Sortie attendue dans la console :

```
[Node-6001] Bloc #0 (genesis) miné — récompense: 50 BTC → Alice
[Node-6001] Tx diffusée : Alice → Bob (10 BTC) [gossip → 6002, 6003]
[Node-6002] Tx reçue et validée : Alice → Bob (10 BTC)
[Node-6003] Tx reçue et validée : Alice → Bob (10 BTC)
[Node-6002] Bloc #1 miné — 1 transaction confirmée
[Node-6004] Synchronisation : 2 blocs téléchargés depuis Node-6001
[Node-6001] Double dépense détectée — transaction rejetée
```

Module 6 — Fonctionnalités avancées (4e semaine, optionnel)

Ce module est optionnel. Il compte comme bonification (+15% plafonné à 100%). Référence cours : Chap. 5 — Stocker et Utiliser les Bitcoins.

6.1 Multisignatures M-of-N (Diapos 10, Chap. 5)

Implémentez les scripts multisig : une transaction ne peut être dépensée que si K signatures parmi N clés autorisées sont fournies.

```
// Exemple 2-of-3 : Alice, Bob, et Charlie partagent un compte
// La transaction est valide si au moins 2 d'entre eux signent
MultiSigAddress addr = new MultiSigAddress(2,
    List.of(alice.getPublicKey(), bob.getPublicKey(), charlie.getPublicKey()));
Transaction tx = wallet.createMultiSigTx(addr, 5_00000000L);
tx.sign(alice.getPrivateKey());
tx.sign(bob.getPrivateKey());
assert utxoSet.isValid(tx); // 2 signatures sur 3 = OK
```

6.2 Partage de secret de Shamir (Diapos 8–9, Chap. 5)

Implémentez SecretSharing permettant de diviser une clé privée en N fragments dont K suffisent à la reconstruire :

1. `split(BigInteger secret, int k, int n) → List<BigInteger[]>` — génère N points sur un polynôme de degré K-1 dans GF(p).
2. `reconstruct(List<BigInteger[]> shares) → BigInteger` — interpolation de Lagrange avec K fragments.
3. Démontrez que K-1 fragments ne révèlent rien du secret.

6.3 Preuve de Réserves (Diapo 12, Chap. 5)

Implémentez la vérification de solvabilité d'une place d'échange via un arbre de Merkle des soldes clients :

1. Chaque feuille = SHA-256(userId || balance).
2. La racine de Merkle est publiée publiquement.
3. Un client peut vérifier que son solde est inclus sans connaître les autres.

⚠ Attention : Module 6 : la qualité de la démonstration et du rapport compte autant que le code. Un module 6 partiel mais bien expliqué sera mieux noté qu'un module 6 complet mais sans explication.

Livrables attendus

À remettre sur Moodle avant la date limite :

#	Livrable	Format	Poids
L1	Code source Java complet (Maven multi-modules)	ZIP ou dépôt Git	50%
L2	Rapport technique	PDF, 10–15 pages	25%
L3	Démonstration en classe	15 min + 5 min questions	20%
L4	Tests unitaires (JUnit 5)	inclus dans L1	5%

Contenu du rapport (L2)

- Page de titre : noms, matricules, titre du projet.
- Introduction (1/2 page) : contexte et objectifs.
- Architecture globale : diagramme de packages/classes principal.
- Description de chaque module : choix de conception, difficultés rencontrées.
- Résultats des tests : captures d'écran ou logs de la démonstration.

- Analyse critique : limites de votre implémentation par rapport au vrai Bitcoin.
- Conclusion et apprentissages.
- Références (cours IFT605, Nakamoto 2008, etc.).

Contenu de la démonstration (L3)

- Lancer les 3 nœuds locaux et démontrer la diffusion gossip.
- Effectuer une transaction Alice → Bob et la faire confirmer (minage).
- Démontrer la propriété tamper-evident (modification détectée).
- Démontrer la preuve de Merkle (appartenance d'une transaction).
- Simuler une tentative de double dépense et montrer le rejet.
- (Module 6) Démontrer multisig ou partage de secret si implémenté.

Calendrier recommandé

Semaine	Dates	Travail à réaliser	Jalon
Semaine 1	Jours 1–7	Module 1 (crypto) + Module 2 (blockchain + Merkle). Mise en place Maven multi-modules.	Tests unitaires Modules 1 & 2 passent (green)
Semaine 2	Jours 8–14	Module 3 (UTXO + scripts) + Module 4 (PoW + double dépense). Intégration Modules 1→4.	Simulation double dépense fonctionnelle
Semaine 3	Jours 15–21	Module 5 (P2P + portefeuille). Scénario demo/ intégré. Rapport. Répétition démo.	Démo complète 3 nœuds + remise L1 + L2
Semaine 4 (optionnel)	Jours 22–28	Module 6 : multisig, partage de secret, preuve de réserves.	Remise finale avec bonification

i Note : Conseil : commencez le module 2 (blockchain) dès que le module 1 est stable — ils sont fortement couplés. Le module 5 (P2P) est le plus chronophage : prévoyez 2 jours supplémentaires si c'est la première fois que vous programmez des sockets Java.

Critères d'évaluation

Critère	Détail	Points
---------	--------	--------

Module 1 — Crypto	SHA-256 correct, ECDSA sign/verify, commitment scheme	12
Module 2 — Blockchain	Arbre de Merkle, preuve d'appartenance, tamper-evident	18
Module 3 — Transactions	UTXO set correct, vérification des balances, script P2PKH	18
Module 4 — Consensus	PoW fonctionnel, difficulté ajustable, simulation double dépense	18
Module 5 — P2P + Wallet	Gossip entre ≥ 3 nœuds, portefeuille, scénario intégré	14
Tests unitaires	≥ 3 tests JUnit 5 par module, cas normaux et cas limites	5
Rapport	Clarté, profondeur, analyse critique, références	10
Démo	Fluidité, capacité à répondre aux questions, live coding possible	5
Module 6 (bonification)	Multisig + partage de secret + preuve de réserves	+15 (plafonné à 100)
	TOTAL	100

⚠ Attention : Pénalités :

- Retard de remise : -5% par jour calendrier.
- Code qui ne compile pas : Module concerné noté 0.
- Plagiat entre équipes : note 0 pour les deux équipes + signalement.
- Librairie Bitcoin externe (non autorisée) : -50% sur le module concerné.

Ressources et références

Présentations du cours (Moodle IFT605)

- Chap. 1 — Chaînes de blocs et Bitcoin (introduction)
- Chap. 2 — Bitcoin et Cryptographie (SHA-256, Merkle, ECDSA)
- Chap. 3 — Bitcoin et la Décentralisation (consensus, PoW, attaques)
- Chap. 4 — Bitcoin Mécanismes (UTXO, scripts, réseau P2P)
- Chap. 5 — Stocker et Utiliser les Bitcoins (wallets, multisig, secret sharing)

Références techniques

- S. Nakamoto, « Bitcoin: A Peer-to-Peer Electronic Cash System », 2008.
<https://bitcoin.org/bitcoin.pdf>

- A. Narayanan et al., Bitcoin and Cryptocurrency Technologies, Princeton, 2016. (Chapitres 1–5 correspondent exactement aux modules du projet.)
- BIP-0032 — Hierarchical Deterministic Wallets : <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>
- Java Cryptography Architecture (JCA) : <https://docs.oracle.com/en/java/javase/21/security/>
- Bouncy Castle (ECDSA secp256k1) : <https://www.bouncycastle.org/java.html>

Outils recommandés

Outil	Usage	URL
IntelliJ IDEA / Eclipse	IDE Java	https://www.jetbrains.com/idea/
Maven 3.9.x	Build + modules	https://maven.apache.org/
JUnit 5	Tests unitaires	https://junit.org/junit5/
Bouncy Castle	ECDSA secp256k1	https://www.bouncycastle.org/
Wireshark	Observer le P2P	https://www.wireshark.org/
Bitcoin Explorer	Référence blocs	https://blockstream.info/